End-to-End MLOps for Heart Disease Classification

BITS Pilani WILP | AIMLCZG523 — Machine Learning Operations | Assignment 01 (2026)

Submission report draft — execution evidence and actual results must be inserted before final submission.

Project scope: reproducible data acquisition and EDA, feature engineering, two-model classification, cross-validation, MLflow tracking, model packaging, automated tests, CI/CD, Docker, deployment, monitoring, and reporting.

Status: source files and validation scaffolding have been generated. The official UCI data download and production execution were unavailable in the report-generation environment. No real model metrics or deployment success are claimed.

1. Executive Summary

This project is designed to demonstrate a complete MLOps lifecycle for a binary classification task using the Cleveland subset of the UCI Heart Disease dataset. The output is an API that accepts a structured patient-feature payload and returns a binary model prediction and class probabilities. The system includes reproducible preprocessing, Logistic Regression and Random Forest candidates, cross-validation, experiment tracking, testing, a container definition, deployment manifests, and basic logging.

The project is educational only. It is not a medical device, has not been externally or clinically validated, and must not be used to diagnose, triage, or treat any person.

Execution status: implementation files have been generated and twelve automated software tests passed using synthetic fixtures. Official-dataset metrics, MLflow runs, remote CI execution, Docker build, cloud deployment, screenshots, and demo video remain pending until run in a suitable external environment.

2. Problem Statement and Objectives

The assignment asks for a reproducible machine-learning solution that predicts the presence or absence of heart disease from structured clinical attributes and exposes the resulting model through an API. The MLOps objective is not only to train a classifier but also to track experiments, package the pipeline, automate quality checks, containerize inference, deploy it, and observe its operation.

- Acquire and document the UCI dataset.
- Inspect and clean data, including missing values and target mapping.
- Train and compare at least two classifiers.
- Track parameters, metrics, and artifacts using MLflow.
- Package preprocessing and the classifier as one reusable pipeline.
- Automate linting, tests, and training in CI.
- Expose JSON inference through a containerized API.
- Provide deployment manifests, logging, and monitoring configuration.

Success criteria are the nine rubric items in the official assignment. All final claims must be supported by actual execution evidence.

3. Dataset and Data Preparation

The selected source is the UCI Machine Learning Repository Heart Disease dataset. The processed Cleveland subset is commonly described as 303 instances with 13 predictors and one diagnosis field. The original diagnosis field `num` takes values 0 through 4. This implementation maps 0 to absence and values 1–4 to presence, yielding a binary target.

The raw processed file contains no header; the acquisition code assigns the documented 14 column names and interprets `?` as missing. Required columns are checked before model training. Numeric values are converted explicitly, rows with an unknown target are removed, and feature missingness is handled inside the preprocessing pipeline to avoid fitting imputers on the test data.

Dataset source: https://archive.ics.uci.edu/dataset/45/heart+disease

Actual row counts, missing-value counts, and class distribution: PENDING execution on the official dataset. Insert the generated `reports/figures/eda_summary.json` results here after execution.

4. Exploratory Data Analysis

The EDA script generates a target class-balance plot, feature histograms, a feature-correlation heatmap, descriptive statistics, and a JSON summary. The plots are intended to make class imbalance, feature ranges, possible outliers, and linear associations easier to inspect.

Interpretation must be based on the generated plots, not assumptions. The class balance informs the use of stratified splitting and balanced model weights. Correlation is descriptive and does not establish causation. The small historical sample also limits generalization.

Evidence to insert: class_balance.png, feature_histograms.png, correlation_heatmap.png, and the actual EDA summary. No plots from the official dataset are included in this draft because the dataset could not be downloaded in the build environment.

5. Feature Engineering and Model Development

The feature pipeline uses a ColumnTransformer. Numeric features are imputed with their training-fold median and scaled with StandardScaler. Categorical or discrete-coded features are imputed with the most frequent training-fold value and one-hot encoded with unknown categories ignored. These transformers are fitted as part of each model pipeline, reducing leakage risk and ensuring training/inference consistency.

Two candidates are defined: Logistic Regression with balanced class weights, and Random Forest with balanced class weights. A stratified 80/20 holdout split is created with a fixed seed. Each candidate is tuned with GridSearchCV using stratified five-fold cross-validation: Logistic Regression searches C values of 0.1, 1, and 10; Random Forest searches tree counts of 150/300, maximum depth of None/8, and minimum leaf sizes of 1/2. ROC-AUC selects the best parameter set and then the candidate model. The holdout test set is reserved for final evaluation.

Accuracy, precision, recall, and ROC-AUC are calculated for cross-validation and the held-out test split. The selected model is the candidate with the highest mean cross-validation ROC-AUC; the holdout test metrics are reported only after selection.

Actual metrics and model selection: PENDING execution. Populate this section from `artifacts/metrics.json`; do not estimate or fabricate values.

6. Experiment Tracking and Reproducibility

When MLflow is installed, each candidate run logs the model name, fixed random seed, holdout proportion, cross-validation fold count, and the calculated metrics. The fitted pipeline is logged as a model artifact. The best selected pipeline is saved to `artifacts/model.joblib`, with metrics in JSON and a model card describing intended use and limitations.

The requirements file defines project dependencies. A clean environment should install these dependencies before running the download, EDA, training, and API commands. For persistent cloud tracking, configure a remote MLflow tracking URI and artifact store; a temporary local runtime may not preserve tracking files after shutdown.

Evidence required: MLflow UI showing two runs, parameters, metrics, and model artifacts; artifact file listing; successful model reload and prediction. These are pending external execution.

7. Testing, CI/CD, API, and Containerization

Twelve Pytest tests passed in the build environment. They validate dataset schema checks, binary target validation, model pipeline fitting/prediction, API health behavior without a model artifact, invalid request handling, controlled 503 behavior when the model is missing, Prometheus text metrics, and a successful prediction using a model fitted on synthetic fixtures. Synthetic fixtures validate software behavior only; they are not a substitute for training and evaluating on the UCI dataset.

The GitHub Actions workflow installs dependencies, runs Ruff linting and Pytest, downloads the official dataset (with a validated public mirror fallback), executes model training, builds the Docker image, smoke-tests the container health and metrics endpoints, and uploads artifacts. On pushes to main/master it is configured to publish the tested image to GitHub Container Registry using the repository token. This workflow must be pushed and run in an actual repository before its success can be claimed.

The FastAPI service exposes `/health`, `/metrics`, and `/predict`. Prediction requests are schema-validated, unknown fields are rejected, and the response includes the binary label, model confidence score, and class probabilities. The confidence score is not a clinically calibrated probability.

The Dockerfile packages the API and generated model artifact. A Docker build must occur after training has produced the model file. Actual remote workflow status and Docker build/run proof remain pending.

8. Deployment, Monitoring, and Security

A Kubernetes Deployment and LoadBalancer Service manifest are included as a starting point. Before applying them, the placeholder image name must be replaced with a real container image in a registry accessible to the cluster. The service type and access method depend on the selected environment; a cloud LoadBalancer is not automatically available in local Kubernetes.

The API logs request method, path, status, and elapsed time, returns a processing-time header, and exposes Prometheus text metrics for request counts, cumulative latency, and model-loaded status. A scrape configuration and Grafana dashboard JSON are included. These process-local metrics reset on restart, so production monitoring requires persistent storage and alerting.

Security measures include avoiding secrets in source control, rejecting unexpected request fields, limiting plausible feature ranges, and avoiding sensitive patient-data logging. These measures do not establish clinical suitability or comprehensive production security.

Deployment evidence to insert: actual deployment status, health response, prediction response, service endpoint, and monitoring output. None is claimed complete in this draft.

9. Architecture and Reproduction

Architecture: UCI data → validation and EDA → preprocessing → tuned Logistic Regression / Random Forest → MLflow and model artifact → GitHub Actions CI → FastAPI/Docker → Kubernetes or cloud → Prometheus metrics and request logs.

Reproduction: install requirements, download UCI data, run EDA and tuned training, inspect MLflow and metrics, run tests, build/smoke-test Docker, then deploy the image and verify endpoints. See README.md for commands.

Repository URL: PENDING creation and publication of a real GitHub repository.

Deployed API URL/access instructions: PENDING actual deployment.

Video URL/file: PENDING actual recording.

10. Limitations, Conclusion, and Final Evidence Checklist

The selected dataset is small and historical, and its population may not represent current or diverse clinical settings. The model has no clinical validation and must not be used for medical decisions. The evaluation is intended to demonstrate software and MLOps practices rather than clinical performance.

Conclusion: the project structure and code implement the intended workflow and twelve software-level tests passed. This report remains a draft until real-dataset execution, MLflow tracking, Docker build/run, remote CI, deployment, monitoring dashboard, screenshots, and video evidence have been added. Synthetic tests must not be presented as model validation.

- [ ] Official dataset acquired and EDA run.
- [ ] Actual two-model metrics and selection rationale inserted.
- [ ] MLflow runs and artifacts verified.
- [ ] Clean environment/reproducibility verified.
- [ ] Pytest and lint results captured.
- [ ] GitHub Actions run verified.
- [ ] Docker build and prediction verified.
- [ ] Kubernetes/cloud deployment verified.
- [ ] Monitoring output captured.
- [ ] GitHub URL, endpoint URL/access instructions, screenshots, and video added.

Dataset citation: Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1989). Heart Disease. UCI Machine Learning Repository. https://doi.org/10.24432/C52P4X

